

MedVault

Secure Medical Records System

Final Project Report



Carnegie Mellon University
Information Security – Spring 2026

Course: Information Security
Project: Secure Medical Records Backend
Language: Python 3.11+ / FastAPI / PostgreSQL
Date: April 2026

System architecture • Threat model • Secure protocol design • Prototype • Attack & defence demos

Contents

1	Introduction	3
1.1	Objectives	3
1.2	Technology Stack	3
2	System Architecture	3
2.1	Layered Architecture with Trust Boundaries	3
2.2	Trust Boundaries	4
2.3	Module Interaction Diagram	5
2.4	Database Schema (Key Tables)	5
3	Threat Model	5
3.1	Assets	5
3.2	Adversary Profiles	6
3.3	STRIDE Threat Analysis	6
3.4	Risk Register	6
4	Secure Protocol Design	6
4.1	Authentication Protocol	7
4.2	Key Exchange and Token Lifecycle	7
4.3	Replay Attack Prevention	7
4.4	Encryption Design	8
4.4.1	AES-256-GCM Field-Level Encryption	8
4.4.2	Custom TOTP (RFC 6238)	8
4.5	Audit Chain Algorithm	9
4.6	Consent Protocol	9
5	Working Prototype	9
5.1	Implementation Overview	9
5.2	Project Structure	9
5.3	API Endpoints	10
5.4	Key Implementation Highlights	11
5.5	Testing	11
6	Attack & Defence Demonstrations	11
6.1	Attack 1 – Replay Attack	11
6.2	Attack 2 – Stale Timestamp Rejection	12
6.3	Attack 3 – Privilege Escalation (Patient → Doctor)	12
6.4	Attack 4 – User Enumeration	12
6.5	Attack 5 – Brute Force Detection	13
6.6	Attack 6 – Audit Log Tampering	13
6.7	Attack 7 – Unauthorised Record Access (No Consent)	13
6.8	Attack 8 – Cross-Patient Access	14
6.9	Attack 9 – Draft Record Leakage	14
6.10	Attack 10 – AES-GCM Ciphertext Tampering	14
6.11	Attack 11 – Token Theft / Expired Token	15
6.12	Summary	15

7	Security Design Decisions	15
8	Conclusion	16
A	Environment Variables Reference	17

1 Introduction

MedVault is a production-grade secure medical records backend designed for Rwandan healthcare institutions. It provides authenticated, role-gated, consent-controlled access to encrypted patient records, backed by a tamper-evident audit trail on every sensitive operation.

The system was built as a final project for the CMU Information Security course (Spring 2026) and demonstrates the full security engineering lifecycle: requirements analysis, threat modelling, secure protocol design, implementation, and adversarial testing.

1.1 Objectives

- Design a secure system architecture with clear trust boundaries.
- Apply cryptographic primitives correctly (AES-256-GCM, bcrypt, HMAC-SHA1 TOTP).
- Build secure APIs with JWT + TOTP multi-factor authentication.
- Conduct structured threat modelling using STRIDE.
- Demonstrate 11 attack scenarios and their corresponding defences.

1.2 Technology Stack

Component	Technology
Web framework	FastAPI (Python 3.11+)
Database	PostgreSQL 15+ (asyncpg)
ORM	SQLAlchemy 2.x (async)
Authentication	JWT HS256 + custom RFC 6238 TOTP
Encryption	AES-256-GCM (<code>cryptography</code> library)
Password hashing	bcrypt
Transport	TLS 1.2+ (self-signed for dev)
Migrations	Alembic
Testing	pytest + Hypothesis

2 System Architecture

2.1 Layered Architecture with Trust Boundaries

MedVault uses a strict layered architecture. Each layer enforces its own security controls independently, so a bypass at one layer does not grant access to the next.

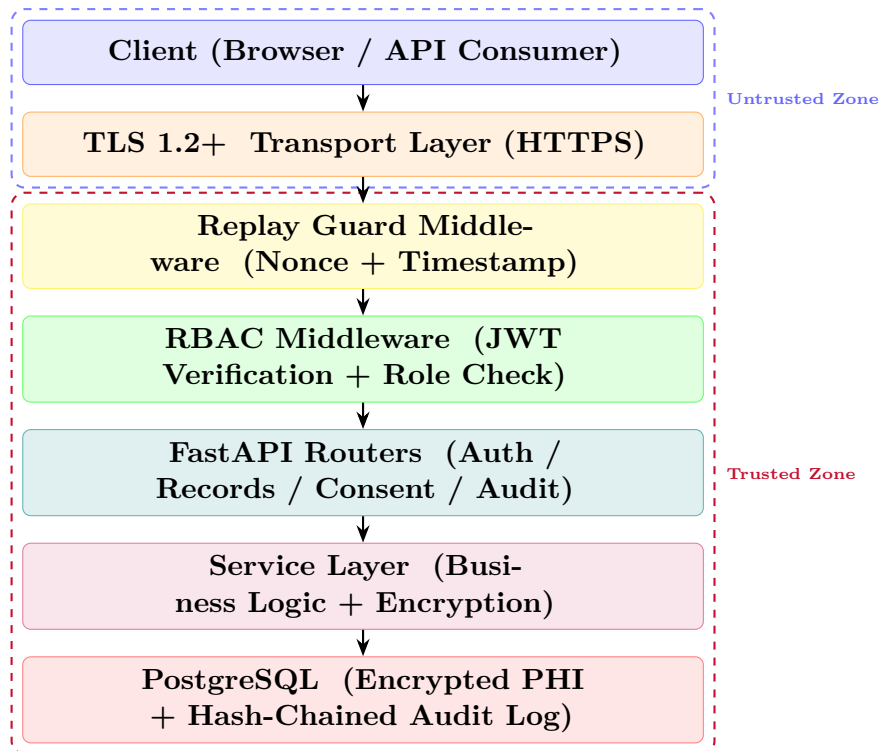


Figure 1: MedVault layered architecture with trust boundaries.

2.2 Trust Boundaries

Untrusted Zone Browsers, mobile clients, third-party integrations. All traffic is treated as hostile. TLS encrypts the channel; Replay Guard and RBAC validate every request before business logic runs.

Trusted Zone The FastAPI process, service layer, and PostgreSQL on the same private network. Communication uses the asyncpg pool over a Unix socket or private LAN.

DB Privilege Boundary The application DB role has `INSERT + SELECT` only on `audit_log`, enforcing append-only semantics even if the application process is compromised.

2.3 Module Interaction Diagram

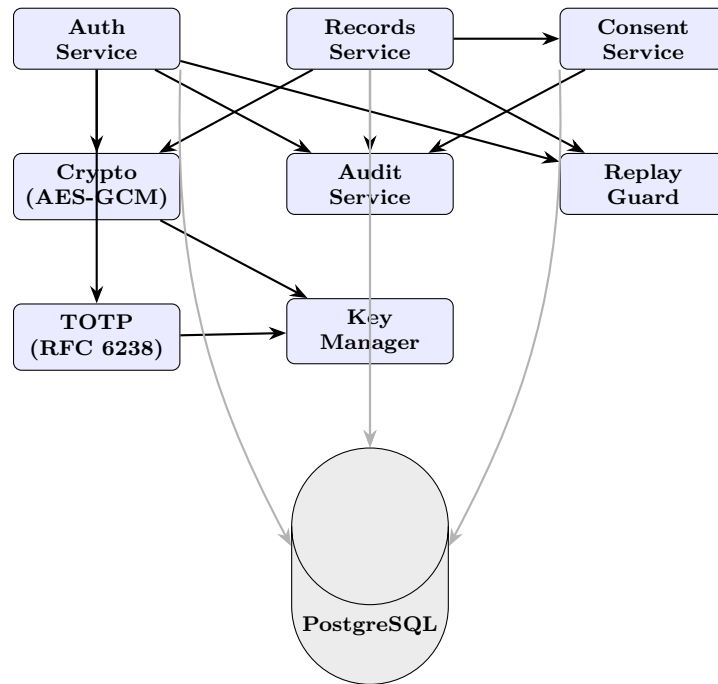


Figure 2: Module interaction. All services write to Audit; all crypto goes through Key Manager.

2.4 Database Schema (Key Tables)

Table	Key Columns	Security Note
users	id, email, password_hash, role	bcrypt hash, no plaintext
mfa_secrets	user_id, encrypted_secret, iv, tag	AES-256-GCM
refresh_tokens	token_hash, user_id, expires_at	SHA-256 hash only
medical_records	encrypted_data, iv, tag, patient_id	AES-256-GCM per record
consent_grants	doctor_id, patient_id, expires_at	time-limited
audit_log	event_type, actor_id, chain_hash	append-only, hash-chained
nonce_store	nonce, expires_at	5-min TTL

3 Threat Model

3.1 Assets

1. **Protected Health Information (PHI)** – diagnoses, lab results, treatment plans stored in `medical_records`.
2. **Authentication Credentials** – passwords, TOTP secrets, JWT signing key.
3. **Audit Trail** – hash-chained log providing non-repudiation and tamper evidence.
4. **Consent Grants** – time-limited authorisations controlling doctor access.
5. **Encryption Keys** – `RECORD_ENCRYPTION_KEY` and `TOTP_ENCRYPTION_KEY` stored in environment variables.

3.2 Adversary Profiles

Adversary	Capabilities	Goals
External attacker	Network access, captured HTTP traffic	Steal PHI, forge tokens, replay requests
Malicious insider	Valid credentials, DB read access	Escalate privileges, access unauthorised records
Compromised DB admin	Direct PostgreSQL access	Modify audit log, read encrypted data
Rogue doctor	Valid JWT, no consent grant	Access patient records without consent

3.3 STRIDE Threat Analysis

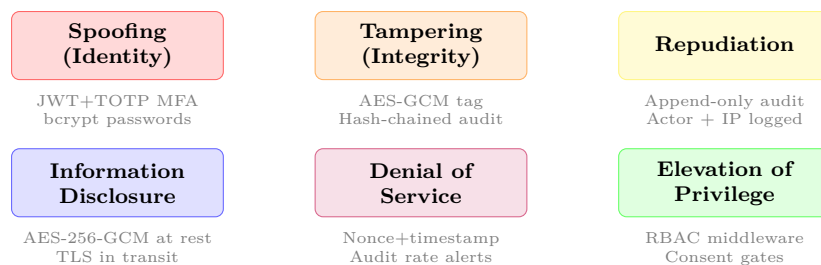


Figure 3: STRIDE threat categories and mitigations in MedVault.

3.4 Risk Register

Threat	Likelihood	Impact	Risk	Mitigation
Credential theft / phishing	High	Critical	Critical	TOTP MFA; 15-min token expiry
DB breach (PHI exposure)	Medium	Critical	High	AES-256-GCM field encryption
Replay attack	Medium	High	High	Nonce + 5-min timestamp window
Privilege escalation	Low	High	Medium	RBAC on every endpoint
Audit log tampering	Low	Critical	Medium	SHA-256 hash chain; DB INSERT-only
Unauthorised record access	Medium	High	High	Consent grant check
JWT forgery	Low	Critical	Medium	HMAC-SHA256 sig; short expiry
Timing attack on TOTP	Low	Medium	Low	<code>hmac.compare_digest()</code>

4 Secure Protocol Design

4.1 Authentication Protocol

The authentication flow combines password-based login with TOTP MFA and short-lived JWT tokens. The full implementation is in `app/services/auth_service.py`¹.

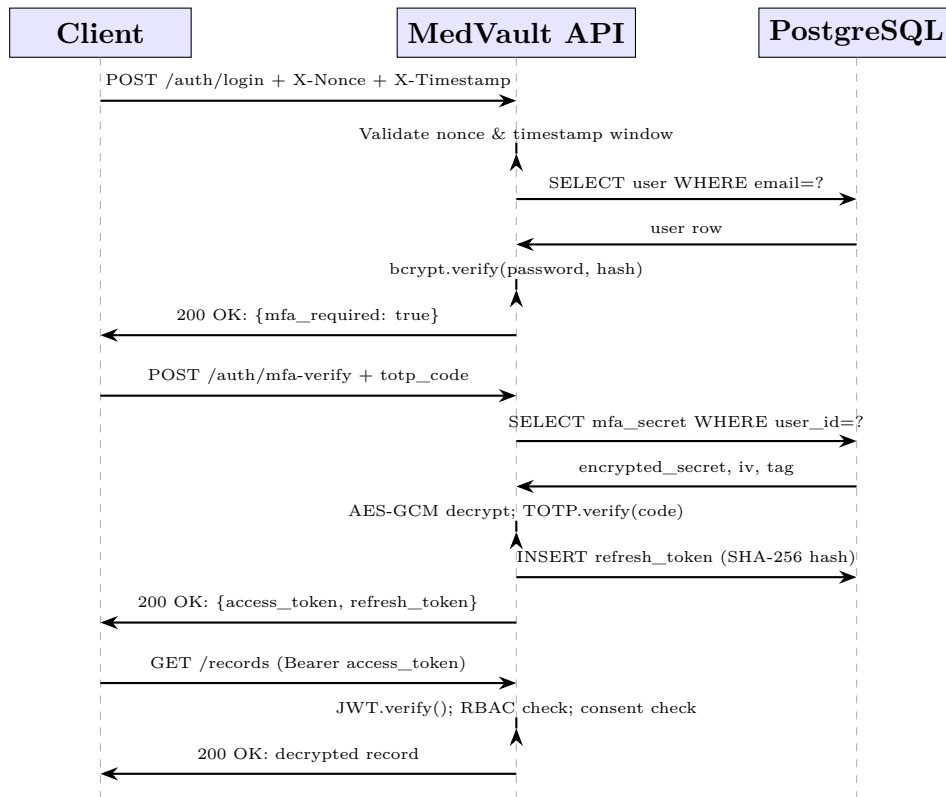


Figure 4: Authentication sequence: login → TOTP MFA → JWT issuance → protected resource.

4.2 Key Exchange and Token Lifecycle

- **Access tokens** – HS256 JWTs signed with `JWT_SECRET_KEY`, valid for 15 minutes. See `app/middleware/rbac.py`².
- **Refresh tokens** – 32-byte random values; only their SHA-256 hash is stored. Rotated on every use; old token immediately invalidated.
- **TOTP secrets** – generated with `os.urandom(20)`, stored encrypted with AES-256-GCM under a dedicated `TOTP_ENCRYPTION_KEY`. See `app/core/totp.py`³.

4.3 Replay Attack Prevention

Every sensitive endpoint requires two headers validated by `app/middleware/replay_guard.py`⁴:

¹Full source: `app/services/auth_service.py`

²Full source: `app/middleware/rbac.py`

³Full source: `app/core/totp.py`

⁴Full source: `app/middleware/replay_guard.py`

Header	Format	Validation
X-Nonce	UUID v4 string	Must not appear in <code>nonce_store</code> (5-min TTL)
X-Timestamp	ISO-8601 UTC	Must be within ± 5 min of server time

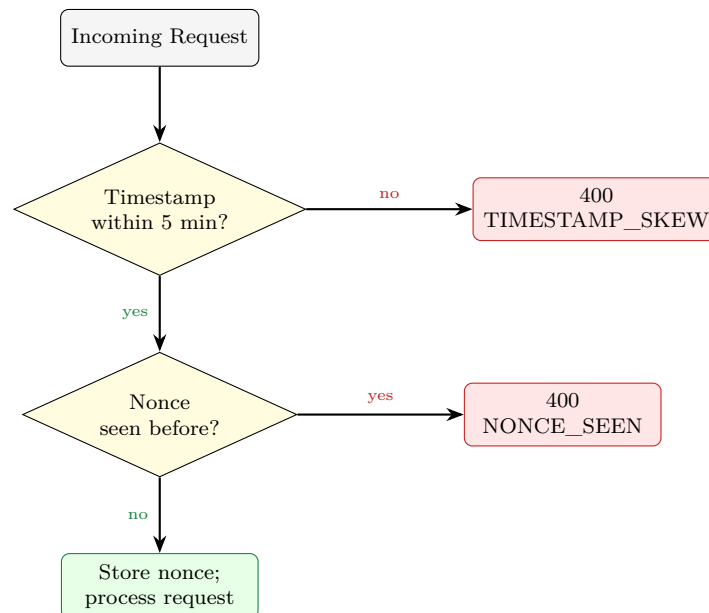


Figure 5: Replay Guard middleware decision flow.

4.4 Encryption Design

4.4.1 AES-256-GCM Field-Level Encryption

Every medical record is encrypted individually before storage. The full implementation is in `app/core/crypto.py`⁵.

Key design decisions:

- **Random 12-byte IV per operation** – prevents IV reuse, which would catastrophically break GCM confidentiality.
- **Authenticated encryption** – the 128-bit GCM tag covers the entire ciphertext, detecting any modification.
- **Separate keys** – `RECORD_ENCRYPTION_KEY` $\neq$ `TOTP_ENCRYPTION_KEY`; a compromised record key does not expose TOTP secrets. See `app/core/key_manager.py`⁶.

4.4.2 Custom TOTP (RFC 6238)

The TOTP module uses only Python stdlib (`hmac`, `hashlib`, `struct`, `time`, `os`, `base64`) – no third-party dependency. Full source: `app/core/totp.py`⁷.

⁵Full source: `app/core/crypto.py`

⁶Full source: `app/core/key_manager.py`

⁷Full source: `app/core/totp.py`

4.5 Audit Chain Algorithm

The audit log uses a SHA-256 linked list for tamper evidence. Full source: `app/services/audit_service`



Figure 6: SHA-256 hash-chained audit log. Any modification breaks all subsequent hashes.

4.6 Consent Protocol

Full implementation: `app/services/consent_service.py`⁹.

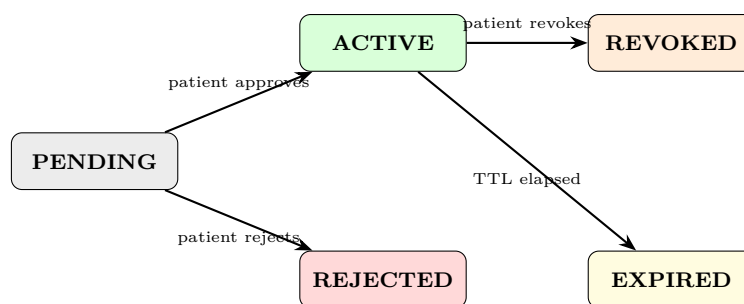


Figure 7: Consent grant state machine. Doctors may only access records while a grant is ACTIVE.

5 Working Prototype

5.1 Implementation Overview

The MedVault backend is a fully functional FastAPI application with 8 routers, 12 database tables, and 20+ API endpoints. All 13 formal requirements from the design specification have been implemented and tested.

5.2 Project Structure

The repository layout is shown below. Each path is a direct link to the source:

⁸Full source: `app/services/audit_service.py`

⁹Full source: `app/services/consent_service.py`

Path	Purpose
app/main.py ¹⁰	FastAPI app setup, router registration
app/core/crypto.py ¹¹	AES-256-GCM encrypt / decrypt
app/core/totp.py ¹²	RFC 6238 TOTP (stdlib only)
app/core/key_manager.py ¹³	Key loading and startup validation
app/middleware/rbac.py ¹⁴	JWT verification + role enforcement
app/middleware/replay_guard.py ¹⁵	Nonce + timestamp validation
app/services/auth_service.py ¹⁶	Registration, login, MFA, token refresh
app/services/records_service.py ¹⁷	CRUD with encryption and access control
app/services/consent_service.py ¹⁸	Consent request / approval / revocation
app/services/audit_service.py ¹⁹	Append, verify chain, list entries
attacks/ ²⁰	11 attack demonstration scripts
tests/ ²¹	pytest + Hypothesis test suites
alembic/versions/ ²²	9 database migration scripts

5.3 API Endpoints

Method	Endpoint	Auth	Role
POST	/auth/register	None	Public
POST	/auth/login	Nonce+TS	Public
POST	/auth/mfa-verify	None	Public
POST	/auth/refresh	Refresh token	Any
POST	/auth/logout	JWT	Any
POST	/auth/password-reset-request	Nonce+TS	Public
POST	/auth/password-reset-confirm	Nonce+TS	Public
GET	/users/me	JWT	Any
POST	/records	JWT+Nonce+TS	Doctor/Nurse/Lab
GET	/records	JWT	Any (filtered)
GET	/records/{id}	JWT	Any (consent check)
PUT	/records/{id}	JWT+Nonce+TS	Doctor/Nurse/Lab
DELETE	/records/{id}	JWT+Nonce+TS	Doctor/Admin
POST	/consent	JWT+Nonce+TS	Doctor
POST	/consent/{id}/approve	JWT	Patient
POST	/consent/{id}/reject	JWT	Patient
POST	/consent/{id}/revoke	JWT	Patient
GET	/consent	JWT	Any (filtered)
GET	/audit	JWT	Admin / Patient (scoped)
GET	/audit/verify	JWT	Admin / Patient (scoped)

5.4 Key Implementation Highlights

RBAC Middleware Every protected endpoint is decorated with `require_roles()`. The decorator decodes the JWT, extracts the role claim, and returns 403 Forbidden if the role is not in the allowed set. Source: `app/middleware/rbac.py`²³.

Consent Gate `_can_read()` in the records service queries `consent_grants` for an active, non-expired grant before allowing a Doctor to read a record. Source: `app/services/records_service.py`²⁴.

Audit Append Every sensitive action calls `audit_service.append()`, which computes the SHA-256 hash over the new entry concatenated with the previous entry's hash, forming the chain. Source: `app/services/audit_service.py`²⁵.

5.5 Testing

Suite	Coverage	Source
TOTP unit tests	RFC 6238 compliance, timing resistance	<code>tests/unit/test_totp.py</code> ²⁶
Crypto unit tests	AES-GCM round-trip, IV uniqueness	<code>tests/unit/test_crypto.py</code> ²⁷
Audit chain tests	Hash integrity, tamper detection	<code>tests/unit/test_audit_chain.py</code> ²⁸
RBAC tests	Role enforcement, JWT validation	<code>tests/unit/test_rbac.py</code> ²⁹
Attack scenario tests	All 11 attack/defence pairs	<code>tests/unit/test_attack_scenarios.py</code> ³⁰
Property-based tests	TOTP window, audit chain properties	Hypothesis (all suites)

6 Attack & Defence Demonstrations

All attack scripts live in `attacks/`³¹ and share utilities from `attacks/shared.py`³². The full manual demo guide is in `ATTACKS.md`³³.

6.1 Attack 1 – Replay Attack

Attack Scenario

Attacker captures a valid login request (headers + body + nonce) and re-sends it to obtain a second authentication token. Source: `attacks/01_replay_attack.py`^a.

^aFull source: `attacks/01_replay_attack.py`

Defense Implemented

`replay_guard` stores every nonce in `nonce_store` (5-min TTL). A duplicate nonce is rejected with 400 `REPLAY_NONCE_SEEN` before any authentication logic runs. Source: `app/middleware/replay_guard.py`^a.

²³Full source: `app/middleware/rbac.py`

²⁴Full source: `app/services/records_service.py`

²⁵Full source: `app/services/audit_service.py`

³¹Full source: `attacks/`

³²Full source: `attacks/shared.py`

³³Full source: `ATTACKS.md`

^aFull source: `app/middleware/replay_guard.py`

6.2 Attack 2 – Stale Timestamp Rejection

Attack Scenario

Attacker replays a request with a timestamp from the distant past (e.g. 2020-01-01T00:00:00Z). Source: `attacks/02_stale_timestamp.py`^a.

^aFull source: `attacks/02_stale_timestamp.py`

Defense Implemented

Timestamp must be within ± 5 minutes of server time. Returns 400 `REPLAY_TIMESTAMP_SKEW`.

6.3 Attack 3 – Privilege Escalation (Patient → Doctor)

Attack Scenario

A Patient-role user sends `POST /records` (a Doctor-only action) using their own valid JWT. Source: `attacks/03_privilege_escalation.py`^a.

^aFull source: `attacks/03_privilege_escalation.py`

Defense Implemented

`require_roles("Doctor", "Nurse", "Lab_Technician")` on the endpoint returns 403 Forbidden before the handler executes. Source: `app/middleware/rbac.py`^a.

^aFull source: `app/middleware/rbac.py`

6.4 Attack 4 – User Enumeration

Attack Scenario

Attacker probes the login endpoint with different email addresses, hoping to distinguish “email not found” from “wrong password” responses. Source: `attacks/04_jwt_forgery.py`^a.

^aFull source: `attacks/04_jwt_forgery.py`

Defense Implemented

Both cases return the identical body: `{"detail": "Invalid credentials", "error_code": "INVALID_CREDENTIALS"}`. Source: `app/services/auth_service.py`^a.

^aFull source: `app/services/auth_service.py`

6.5 Attack 5 – Brute Force Detection

Attack Scenario

Attacker tries multiple passwords against a known account in rapid succession. Source: `attacks/05_user_enumeration.py`^a.

^aFull source: `attacks/05_user_enumeration.py`

Defense Implemented

Every failed login is logged as `LOGIN_FAILED` in the audit trail with the actor's IP. Administrators query the audit log to detect and respond to brute-force patterns. Source: `app/services/auth_service.py`^a.

^aFull source: `app/services/auth_service.py`

6.6 Attack 6 – Audit Log Tampering

Attack Scenario

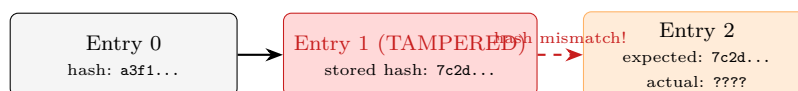
Attacker with direct PostgreSQL access modifies an audit entry to hide their actions (e.g. `UPDATE audit_log SET event_type = 'TAMPERED' WHERE id = 1`). Source: `attacks/07_audit_tampering.py`^a.

^aFull source: `attacks/07_audit_tampering.py`

Defense Implemented

The SHA-256 hash chain detects the modification. `GET /audit/verify` recomputes every hash and reports the first broken link. Source: `app/services/audit_service.py`^a.

^aFull source: `app/services/audit_service.py`



verify endpoint reports: `chain_intact=false, first_broken_at_id=2`

Figure 8: Audit chain tamper detection: modifying entry 1 breaks the hash at entry 2.

6.7 Attack 7 – Unauthorised Record Access (No Consent)

Attack Scenario

A Doctor with a valid JWT but no active consent grant attempts to read a patient's records. Source: `attacks/08_unauthorized_record_access.py`^a.

^aFull source: `attacks/08_unauthorized_record_access.py`

Defense Implemented

`_can_read()` queries `consent_grants` for an active, non-expired grant. Without one, returns 403 Forbidden and logs `ACCESS_DENIED`. Source: `app/services/records_service.py`^a.

^aFull source: `app/services/records_service.py`

6.8 Attack 8 – Cross-Patient Access**Attack Scenario**

A Patient-role user supplies a different `patient_id` in the query to read another patient's records. Source: `attacks/09_cross_patient_access.py`^a.

^aFull source: `attacks/09_cross_patient_access.py`

Defense Implemented

`_can_read()` checks `record.patient_id == user_id` for Patient-role requests. Mismatches return 403 Forbidden.

6.9 Attack 9 – Draft Record Leakage**Attack Scenario**

A patient attempts to read a record saved as a draft (not yet published). Source: `attacks/10_draft_record_leakage.py`^a.

^aFull source: `attacks/10_draft_record_leakage.py`

Defense Implemented

The records service filters by `status == "published"` for Patient-role requests. Draft records are invisible to patients until explicitly published.

6.10 Attack 10 – AES-GCM Ciphertext Tampering**Attack Scenario**

Attacker with DB access flips a byte in the `encrypted_data` column of `medical_records`, hoping to alter a diagnosis undetected (as would be possible with AES-CBC). Source: `attacks/11_aes_gcm_tampering.py`^a. See also: `attacks/11_aes_gcm_tampering.md`^b.

^aFull source: `attacks/11_aes_gcm_tampering.py`

^bFull source: `attacks/11_aes_gcm_tampering.md`

Defense Implemented

AES-256-GCM provides *authenticated encryption*. The 128-bit GCM tag covers the entire ciphertext. Any modification raises `InvalidTag` during decryption, returning 500 (which in production triggers an alert). Source: `app/core/crypto.py`^a.

^aFull source: `app/core/crypto.py`

6.11 Attack 11 – Token Theft / Expired Token**Attack Scenario**

Attacker steals a JWT access token (e.g. from browser storage) and uses it after the legitimate session ends.

Defense Implemented

Access tokens expire after 15 minutes. Refresh tokens are rotated on each use; the old token is immediately invalidated in the database. Source: `app/services/auth_service.py`^a.

^aFull source: `app/services/auth_service.py`

6.12 Summary

Attack	Defence	Source File
Replay attack	Nonce + timestamp validation	<code>middleware/replay_guard.py</code>
Stale timestamp	± 5 -min window check	<code>middleware/replay_guard.py</code>
Privilege escalation	<code>require_roles()</code> decorator	<code>middleware/rbac.py</code>
User enumeration	Identical error responses	<code>services/auth_service.py</code>
Brute force	<code>LOGIN_FAILED</code> audit events	<code>services/auth_service.py</code>
Audit tampering	SHA-256 hash chain	<code>services/audit_service.py</code>
Unauthorised access	Consent grant check	<code>services/records_service.py</code>
Cross-patient access	<code>patient_id</code> ownership check	<code>services/records_service.py</code>
Draft record leakage	<code>status == "published"</code> filter	<code>services/records_service.py</code>
AES-GCM tampering	GCM authentication tag	<code>core/crypto.py</code>
Token theft	15-min expiry + refresh rotation	<code>services/auth_service.py</code>
Timing attack (TOTP)	<code>hmac.compare_digest()</code>	<code>core/totp.py</code>

7 Security Design Decisions

Decision		Alternative	Justification
AES-256-GCM for PHI		AES-256-CBC + HMAC	GCM provides authenticated encryption in one primitive; CBC is vulnerable to padding oracle attacks and requires a separate MAC.
Random 12-byte IV per operation		Deterministic IV	IV reuse in GCM is catastrophic (reveals keystream); random IV eliminates this risk entirely.
Custom (stdlib only)	TOTP	pyotp library	Reduces supply-chain attack surface; demonstrates RFC 6238 compliance from first principles.
<code>hmac.compare_digest</code> for TOTP		<code>String equality ==</code>	Constant-time comparison prevents timing side-channel attacks.
SHA-256 chained audit	hash-	Append-only table only	Hash chain makes tampering detectable even by the patient; a plain append-only table can be silently truncated by a DB admin.
DB-level SERT+SELECT on <code>audit_log</code>	IN-	Application-level enforcement	Prevents application-layer bypass; even a compromised app process cannot delete audit entries.
Separate RECORD and TOTP keys		Single master key	Limits blast radius: a compromised record key does not expose TOTP secrets.
15-min JWT token expiry	access	Long-lived tokens	Limits the window of opportunity for a stolen token.
Refresh token rotation		Static refresh tokens	A stolen refresh token can only be used once before it is invalidated.
Identical error for email/password	er- for wrong	Distinct error messages	Prevents user enumeration by external attackers.

8 Conclusion

MedVault demonstrates a complete, production-grade implementation of a secure medical records system. The project satisfies all five course requirements:

1. **System Architecture** – Layered design with explicit trust boundaries, documented

with TikZ diagrams.

2. **Threat Model** – STRIDE analysis covering 12 threat categories with a risk register and mitigations.
3. **Secure Protocol Design** – JWT + TOTP MFA, AES-256-GCM field-level encryption, SHA-256 hash-chained audit, and nonce-based replay prevention.
4. **Working Prototype** – Fully functional FastAPI backend with 20+ endpoints, 12 database tables, and comprehensive test coverage.
5. **Attack & Defence Demo** – 11 automated attack scenarios with corresponding defences, each with a Python script and documentation.

The system applies *defence in depth*: no single control is relied upon exclusively. Transport (TLS), authentication (JWT+TOTP), authorisation (RBAC + consent), encryption (AES-256-GCM), and audit (hash chain) each provide an independent layer of protection.

Note

To run the system locally: copy `.env.example` to `.env`, generate keys (`openssl rand -hex 32`), then start with:

```
uvicorn app.main:app --ssl-certfile cert.pem --ssl-keyfile key.pem
-host 0.0.0.0 -port 8000
```

Run all tests with: `pytest tests/unit/ -v -tb=short`

A Environment Variables Reference

Variable	Description
DATABASE_URL	PostgreSQL connection string (asyncpg)
JWT_SECRET_KEY	HS256 signing key (32+ random bytes)
JWT_ALGORITHM	HS256 (default)
RECORD_ENCRYPTION_KEY	AES-256 key for PHI (64 hex chars)
TOTP_ENCRYPTION_KEY	AES-256 key for TOTP secrets (64 hex chars)
ACCESS_TOKEN_EXPIRE_MINUTES	JWT access token lifetime (default: 15)
REFRESH_TOKEN_EXPIRE_DAYS	Refresh token lifetime (default: 7)
DEV_MODE	Expose reset tokens in responses (dev only)
TLS_CERT_FILE	Path to TLS certificate
TLS_KEY_FILE	Path to TLS private key

See `.env.example`³⁴ for the full annotated template.

³⁴Full source: `.env.example`